

ISEN Ouest - Electronique Numérique
Travaux Pratiques - Linux Embarqué

Compte Rendu

TP4 - TP5 - TP6

Programmation système sous Linux/Unix en langage C

Matéo LE TOUZIC, Félix MARQUET

4ème année - ISEN Ouest, site de Nantes

Année 2025-2026

Table des matières

TP4 - Duplication de processus	4
Objectifs	4
4.1 - Fonction Fork	4
Observation initiale	4
Affichage des nombres pairs et impairs	4
4.2 - Communication entre processus	4
Principe	4
Fonctionnement	4
TP5 - Signaux	5
Objectifs	5
5.1 - SIGKILL	5
Observation avec ps -ef	5
Terminaison du fils par le père	5
5.2 - Utilisation de la fonction signal	5
Déblocage du père par le fils	5
5.3 - Synchronisation de processus avec les signaux	5
Synchronisation bidirectionnelle	5
5.4 - Attente passive avec sigsuspend	5
Suppression de l'attente active	5
TP6 - Threads : le problème de l'orchestre	6
Objectifs	6
6.2 - Programme initial (sans synchronisation)	6
6.3 - Comptage des spectateurs avec sémaphore mutex	6
Protection de la variable partagée	6
6.4 - Début de concert (DEB_CONCERT)	6
Démarrage conditionnel	6
6.5 - Fin de concert (FIN_CONCERT)	6
Blocage des spectateurs jusqu'à la fin	6
6.6 - Version avec processus	6
Adaptation avec fork et signaux	6
Conclusion	7
Annexes - Code source	8
Annexe A - TP4 : Duplication de processus	8
A.1 - Version 1 : fork de base avec affichage pairs/impairs (exercice 4.1)	8
A.2 - Version 2 : communication par tubes bidirectionnels (exercice 4.2)	8
Annexe B - TP5 : Signaux	11
B.1 - Version 1 : envoi de SIGKILL depuis le père (exercice 5.1)	11
B.2 - Version 2 : déblocage du père par le fils via signal() (exercice 5.2)	11
B.3 - Version 3 : synchronisation bidirectionnelle avec attente active (exercice 5.3)	13
B.4 - Version 4 : attente passive avec sigsuspend (exercice 5.4)	15
Annexe C - TP6 : Threads et l'orchestre	17
C.1 - Version 1 : programme initial sans synchronisation (exercice 6.2)	17

C.2 - Version 2 : comptage avec sémaphore mutex VERROU (exercice 6.3).....	18
C.3 - Version 3 : ajout de DEB_CONCERT pour le démarrage conditionnel (exercice 6.4)	20
C.4 - Version 4 : ajout de FIN_CONCERT, version threads finale (exercice 6.5).....	22
C.5 - Version 5 : adaptation avec processus et signaux (exercice 6.6)	24

TP4 - Duplication de processus

Objectifs

L'objectif de ce TP est de prendre en main les mécanismes de programmation système en langage C sous Linux/Unix. Plus précisément, nous avons abordé la duplication de processus avec l'appel système `fork()` ainsi que la communication inter-processus via des tubes.

4.1 - Fonction Fork

Observation initiale

Après compilation et exécution du programme de base utilisant `fork()`, on observe que le message `printf` est affiché deux fois dans le terminal. Cela est dû au fait que `fork()` crée un processus fils qui est une copie quasi-identique du processus père. Les deux processus reprennent l'exécution juste après l'appel à `fork()` et exécutent donc la même instruction `printf`.

En ajoutant l'affichage du PID via `getpid()`, on peut distinguer les deux processus : le père obtient en retour de `fork()` le PID du fils (valeur positive), tandis que le fils reçoit la valeur 0. Cela permet de différencier les deux branches d'exécution avec une condition sur `pidFork`.

Affichage des nombres pairs et impairs

En utilisant la valeur de retour de `fork()`, on distingue les deux processus : le fils affiche les nombres pairs de 0 à 20, et le père les nombres impairs de 1 à 19. Un appel à `wait(NULL)` dans le père permet d'attendre la fin du fils avant de terminer. On note que l'ordre d'affichage entre les deux processus n'est pas garanti car ils s'exécutent parallèlement.

4.2 - Communication entre processus

Principe

On souhaite synchroniser les deux processus de façon à afficher les nombres dans un ordre défini : le fils affiche d'abord 5 valeurs consécutives, puis le père en affiche 6, et ainsi de suite sur 2 itérations. Pour cela, on utilise deux tubes unidirectionnels (`pipe`) afin d'établir une communication bidirectionnelle :

- `pipe_c2f` : tube du fils vers le père, servant à transmettre la valeur courante du compteur une fois que le fils a terminé son bloc.
- `pipe_f2c` : tube du père vers le fils, servant à transmettre la valeur courante du compteur une fois que le père a terminé son bloc.

Fonctionnement

Le fils commence par afficher ses 5 valeurs, puis écrit la valeur de son compteur dans `pipe_c2f`. Le père lit cette valeur en bloquant sur `read()`, affiche ses 6 valeurs, puis écrit le nouveau compteur dans `pipe_f2c`. Le fils reprend en lisant cette valeur et le cycle recommence. Ce mécanisme assure une synchronisation implicite par les appels bloquants `read()` et `write()`.

TP5 - Signaux

Objectifs

Ce TP approfondit la programmation système Linux avec les signaux UNIX. On y aborde la duplication de processus, l'envoi et la réception de signaux, la suspension de processus, ainsi que la synchronisation via signaux en remplacement d'une attente active.

5.1 - SIGKILL

Observation avec ps -ef

Après ajout d'un `sleep(1)` entre chaque affichage, l'exécution du programme laisse les deux processus actifs pendant plusieurs secondes. En lançant `ps -ef` dans un autre terminal, on observe deux processus distincts avec des PIDs différents correspondant au père et au fils. Pour tuer un processus manuellement, on utilise `kill -9 <PID>`. Le signal 9 (SIGKILL) ne peut pas être intercepté ni ignoré, ce qui garantit la terminaison immédiate.

Terminaison du fils par le père

On modifie le programme pour que le père envoie SIGKILL au fils lorsqu'il atteint le nombre 9. L'appel `kill(pidFork, SIGKILL)` permet d'envoyer ce signal depuis le père. Après l'envoi du signal, `wait(NULL)` est appelé pour récupérer le fils et éviter un processus zombie.

5.2 - Utilisation de la fonction signal

Déblocage du père par le fils

On met en place un mécanisme de déblocage du père par le fils via SIGUSR1. Le père est bloqué en début d'exécution dans une attente active sur la variable globale `ATTENTE_PERE`. Le fils, en fin de sa boucle d'affichage, envoie SIGUSR1 au père via `kill(getppid(), SIGUSR1)`. La fonction `signal()` associe le handler `handler_sigusr1` à ce signal, qui passe `ATTENTE_PERE` à 0 et débloque ainsi le père.

5.3 - Synchronisation de processus avec les signaux

Synchronisation bidirectionnelle

Pour afficher les nombres de 0 à 20 dans l'ordre croissant en alternant fils et père, on met en place une synchronisation bidirectionnelle avec SIGUSR1 et SIGUSR2. Le fils affiche un nombre pair, réarme `ATTENTE_FILS = 1`, envoie SIGUSR1 au père, puis attend en boucle sur `ATTENTE_FILS`. Le père, à réception de SIGUSR1, réarme `ATTENTE_PERE = 1`, affiche le nombre impair suivant, puis envoie SIGUSR2 au fils. On note que l'attente active (boucle `while`) est peu efficace en termes de consommation CPU.

5.4 - Attente passive avec sigsuspend

Suppression de l'attente active

On remplace les boucles `while(ATTENTE_X)` par des appels à `sigsuspend()` qui endorment le processus jusqu'à la réception d'un signal spécifique. On construit deux masques de signaux : `mask_usr1` (tous bloqués sauf SIGUSR1) pour le père, et `mask_usr2` (tous bloqués sauf SIGUSR2) pour le fils. Une précaution importante est prise : SIGUSR2 est bloqué dans le fils dès le début avec `sigprocmask()` pour éviter qu'un signal arrive entre l'envoi de SIGUSR1 et l'appel à `sigsuspend()`, ce qui constituerait une race condition. Les handlers sont des fonctions vides car leur seul rôle est d'interrompre `sigsuspend()`.

TP6 - Threads : le problème de l'orchestre

Objectifs

Ce TP introduit la programmation multi-thread en C avec la bibliothèque POSIX pthread et la synchronisation par sémaphores. Le problème de l'orchestre met en œuvre trois règles de synchronisation introduites progressivement.

6.2 - Programme initial (sans synchronisation)

Dans cette première version, on crée N+1 threads : un thread orchestre et N threads spectateurs. Aucune règle de synchronisation n'est appliquée, les threads s'exécutent librement. Un décalage de 1 seconde est introduit entre la création de chaque thread spectateur. Cette version sert de base de travail pour les questions suivantes.

6.3 - Comptage des spectateurs avec sémaphore mutex

Protection de la variable partagée

Pour afficher le nombre exact de spectateurs dans la salle, on introduit une variable globale CPT et un sémaphore VERROU initialisé à 1 (mutex). L'incrément et la décrémentation de CPT constituent des sections critiques car plusieurs threads pourraient y accéder simultanément. La section critique est réduite au strict minimum : on copie la valeur dans une variable locale juste après la modification, puis on libère immédiatement le verrou avant l'appel à printf(), évitant de bloquer les autres threads pendant l'affichage.

6.4 - Début de concert (DEB_CONCERT)

Démarrage conditionnel

On ajoute un sémaphore DEB_CONCERT initialisé à 0. Le thread orchestre commence par sem_wait(&DEB_CONCERT), ce qui le bloque immédiatement. Lorsqu'un thread spectateur incrémente CPT et constate que local == NMIN (ici 3), il appelle sem_post(&DEB_CONCERT), ce qui débloquent l'orchestre. La vérification est faite sur la variable locale pour éviter de re-acquérir VERROU.

6.5 - Fin de concert (FIN_CONCERT)

Blocage des spectateurs jusqu'à la fin

On ajoute un sémaphore FIN_CONCERT initialisé à 0. Chaque spectateur appelle sem_wait(&FIN_CONCERT) pour se bloquer en attendant la fin du concert. L'orchestre appelle sem_post(&FIN_CONCERT) une seule fois. Pour permettre à tous les spectateurs de se débloquent, chaque spectateur qui reçoit le jeton appelle sem_post(&FIN_CONCERT) avant de sortir, propageant le jeton au suivant. C'est le pattern broadcast par cascade de sem_post. La temporisation de 5 secondes des spectateurs est supprimée dans cette version car les spectateurs attendent de toute façon la fin du concert.

6.6 - Version avec processus

Adaptation avec fork et signaux

On reprend les mêmes règles de synchronisation en remplaçant les threads par deux processus : le père représente l'orchestre et le fils représente l'ensemble des spectateurs. La synchronisation est assurée par SIGUSR1 et SIGUSR2 avec sigsuspend() : le fils envoie SIGUSR1 au père quand NMIN spectateurs sont présents, et le père envoie SIGUSR2 au fils quand le concert est terminé. SIGUSR1 et SIGUSR2 sont bloqués avant le fork() pour éviter la perte d'un signal entre le fork et le premier sigsuspend().

Conclusion

Ces trois TPs ont permis d'aborder progressivement les principaux mécanismes de programmation système sous Linux. Le TP4 a introduit la duplication de processus avec `fork()` et la communication via pipes, mettant en évidence la nécessité de synchroniser les processus pour produire des sorties cohérentes. Le TP5 a approfondi la synchronisation avec les signaux UNIX, en passant d'une attente active peu efficace à une attente passive avec `sigsuspend()`, illustrant l'importance de gérer correctement les race conditions. Enfin, le TP6 a introduit la programmation multi-thread avec `pthread` et les sémaphores POSIX, en résolvant progressivement le problème de l'orchestre, puis en le transposant avec des processus et des signaux.

On retiendra notamment que threads et processus offrent deux approches complémentaires pour la concurrence : les threads partagent la mémoire (d'où la nécessité de mutex), tandis que les processus communiquent via IPC (pipes, signaux), ce qui les rend plus isolés mais plus complexes à synchroniser.

Annexes - Code source

Les fichiers sources sont présentés dans l'ordre chronologique de réalisation, version par version, tels qu'ils ont évolué au fil des exercices. Chaque version porte le numéro de l'exercice correspondant dans le sujet.

Annexe A - TP4 : Duplication de processus

A.1 - Version 1 : fork de base avec affichage pairs/impairs (exercice 4.1)

Première utilisation de fork(). Le fils affiche les nombres pairs et le père les impairs. On identifie les deux processus avec getpid() et on évite les zombies avec wait().

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pidFork;

    /* fork() crée un processus fils, copie quasi-identique du père.
     * Valeur de retour :
     * 0          dans le fils
     * PID fils   dans le père
     * -1         en cas d'erreur */
    pidFork = fork();

    /* Les deux processus (père ET fils) exécutent cette ligne */
    printf("La valeur de retour de fork vaut %d\n", pidFork);
    printf("Le PID du processus courant est: %d\n", getpid());

    if (pidFork == 0) {
        /* Branche fils : pidFork == 0 */
        printf("Je suis dans le fils\n");
        /* Affiche les nombres pairs de 0 à 20 */
        for (int i = 0; i <= 20; i += 2)
            printf("%d ", i);
        printf("\n");
    } else {
        /* Branche père : pidFork == PID du fils */
        printf("Je suis dans le pere\n");
        /* Affiche les nombres impairs de 1 à 19 */
        for (int i = 1; i <= 19; i += 2)
            printf("%d ", i);
        printf("\n");
        /* Attendre la terminaison du fils pour éviter un processus zombie */
        wait(NULL);
    }

    return 0;
}
```

A.2 - Version 2 : communication par tubes bidirectionnels (exercice 4.2)

Remplacement de la version précédente. Deux pipes sont créés pour synchroniser le fils (5 valeurs) et le père (6 valeurs) sur 2 itérations. La synchronisation est assurée implicitement par les appels bloquants read() et write().

```
#include <stdio.h>
#include <unistd.h>
```



```

#include <sys/wait.h>

/* Nombre de valeurs affichées par chaque processus par itération */
#define CHILD_STEPS 5
#define FATHER_STEPS 6
#define ITERATIONS 2

int main() {
    int pipe_c2f[2]; /* tube fils -> père : [0] lecture, [1] écriture */
    int pipe_f2c[2]; /* tube père -> fils : [0] lecture, [1] écriture */

    /* Création des deux tubes de communication bidirectionnelle */
    if (pipe(pipe_c2f) < 0 || pipe(pipe_f2c) < 0) {
        printf("Erreur lors de la creation du pipe\n");
        return 1;
    }

    pid_t pidFork = fork();
    if (pidFork < 0) {
        printf("Erreur lors de la creation du processus fils\n");
        return 1;
    }

    if (pidFork == 0) {
        /* ---- Processus fils ---- */
        /* Fermer les extrémités inutilisées pour éviter les blocages */
        close(pipe_c2f[0]); /* on n'écrit pas depuis c2f[0] */
        close(pipe_f2c[1]); /* on ne lira pas depuis f2c[1] */

        int counter = 0;
        for (int iter = 0; iter < ITERATIONS; iter++) {
            /* Afficher CHILD_STEPS valeurs consécutives */
            for (int i = 0; i < CHILD_STEPS; i++)
                printf("%d ", counter + i);
            printf("\n");
            counter += CHILD_STEPS;

            /* Transmettre la valeur courante du compteur au père */
            write(pipe_c2f[1], &counter, sizeof(counter));

            /* Pour toutes les itérations sauf la dernière,
             * attendre la valeur mise à jour par le père */
            if (iter < ITERATIONS - 1)
                read(pipe_f2c[0], &counter, sizeof(counter));
        }
        close(pipe_c2f[1]);
        close(pipe_f2c[0]);
    } else {
        /* ---- Processus père ---- */
        close(pipe_c2f[1]); /* on n'écrit pas dans c2f[1] */
        close(pipe_f2c[0]); /* on ne lira pas depuis f2c[0] */

        int counter;
        for (int iter = 0; iter < ITERATIONS; iter++) {
            /* Bloquer jusqu'à réception de la valeur envoyée par le fils */
            read(pipe_c2f[0], &counter, sizeof(counter));

            /* Afficher FATHER_STEPS valeurs consécutives */
            for (int i = 0; i < FATHER_STEPS; i++)
                printf("%d ", counter + i);
            printf("\n");
            counter += FATHER_STEPS;

            /* Retransmettre le compteur mis à jour au fils */
            if (iter < ITERATIONS - 1)
                write(pipe_f2c[1], &counter, sizeof(counter));
        }
    }
}

```

```
    }  
    close(pipe_c2f[0]);  
    close(pipe_f2c[1]);  
    /* Attendre la fin du fils avant de quitter */  
    wait(NULL);  
}  
  
return 0;  
}
```

Annexe B - TP5 : Signaux

B.1 - Version 1 : envoi de SIGKILL depuis le père (exercice 5.1)

Le père tue le fils avec `kill(pidFork, SIGKILL)` quand il affiche le nombre 9. `sleep(1)` permet d'observer les deux processus avec `ps -ef`.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>

int main() {
    pid_t pidFork = fork();

    if (pidFork == 0) {
        /* ---- Fils : affiche les nombres pairs ---- */
        for (int i = 0; i <= 20; i += 2) {
            printf("%d ", i);
            sleep(1); /* temporisation pour observer les deux processus avec ps -ef */
        }
    } else {
        /* ---- Père : affiche les nombres impairs ---- */
        for (int i = 1; i <= 19; i += 2) {
            printf("%d ", i);
            sleep(1);
            /* Quand le père atteint 9, il tue le fils avec SIGKILL (signal 9).
             * SIGKILL ne peut pas être intercepté ni ignoré. */
            if (i == 9) {
                kill(pidFork, SIGKILL);
                printf("\nLe fils a été tué par le père\n");
                break;
            }
        }
        /* Récupérer le fils terminé (évite un processus zombie) */
        wait(NULL);
    }

    return 0;
}
```

B.2 - Version 2 : déblocage du père par le fils via signal() (exercice 5.2)

Introduction de `signal()` et d'une variable globale volatile. Le fils débloque le père en fin d'exécution via `SIGUSR1`. Attente active dans le père.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>

/* Variable globale volatile : modifiée par un handler de signal,
 * le mot-clé volatile empêche le compilateur de l'optimiser en cache registre */
volatile int ATTENTE_PERE = 1;

/* Handler SIGUSR1 : débloque le père en passant la variable à 0 */
void handler_sigusr1(int sig) {
    ATTENTE_PERE = 0;
}

int main() {
    pid_t pidFork;
```

```

/* Associer le handler au signal SIGUSR1 avant le fork,
 * les deux processus hériteront de cette association */
signal(SIGUSR1, handler_sigusr1);

/* Le père est bloqué au début en attente active */
/* while (ATTENTE_PERE) ; */

pidFork = fork();

if (pidFork == 0) {
    /* ---- Fils : affiche les nombres pairs ---- */
    for (int i = 0; i <= 20; i += 2) {
        printf("%d ", i);
        sleep(1);
    }
    /* En fin d'exécution, le fils débloquent le père via SIGUSR1 */
    kill(getppid(), SIGUSR1);
} else {
    /* ---- Père : attend le signal SIGUSR1 du fils ---- */
    while (ATTENTE_PERE) ; /* attente active : consomme du CPU */
    printf("Le fils a terminé, le pere continue\n");
    for (int i = 1; i <= 19; i += 2) {
        printf("%d ", i);
        sleep(1);
    }
    printf("\n");
    wait(NULL);
}

return 0;
}

```

B.3 - Version 3 : synchronisation bidirectionnelle avec attente active (exercice 5.3)

Ajout de SIGUSR2 pour synchroniser dans les deux sens. Le fils et le père s'alternent nombre par nombre de 0 à 20. L'attente active sur while() reste présente, elle sera remplacée à l'étape suivante.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>

/* Deux variables de synchronisation :
 *   ATTENTE_PERE = 1 : le père est en attente d'un signal du fils
 *   ATTENTE_FILS = 1 : le fils est en attente d'un signal du père */
volatile int ATTENTE_PERE = 1;
volatile int ATTENTE_FILS = 1;

/* Handler SIGUSR1 : reçu par le père, envoyé par le fils */
void handler_sigusr1(int sig) {
    ATTENTE_PERE = 0;
}

/* Handler SIGUSR2 : reçu par le fils, envoyé par le père */
void handler_sigusr2(int sig) {
    ATTENTE_FILS = 0;
}

int main() {
    pid_t pidFork;

    /* Enregistrement des handlers AVANT le fork pour les deux processus */
    signal(SIGUSR1, handler_sigusr1);
    signal(SIGUSR2, handler_sigusr2);

    pidFork = fork();
    if (pidFork < 0) {
        printf("Erreur lors de la création du processus fils\n");
        return 1;
    }

    printf("La valeur de retour de fork vaut: %d\n", pidFork);
    printf("Le PID du processus courant est: %d\n", getpid());

    if (pidFork == 0) {
        /* ---- Fils : nombres pairs 0, 2, 4, ..., 20 ---- */
        for (int i = 0; i <= 20; i += 2) {
            printf("%d ", i);
            fflush(stdout);
            /* Réarmer le flag avant d'envoyer le signal pour éviter
             * une race condition : le père pourrait répondre
             * avant que le fils ait remis ATTENTE_FILS à 1 */
            ATTENTE_FILS = 1;
            /* Signaler au père qu'il peut afficher son nombre */
            kill(getppid(), SIGUSR1);
            /* Attendre le signal SIGUSR2 du père (sauf après le dernier nombre) */
            if (i < 20)
                while (ATTENTE_FILS) ; /* attente active */
        }
    } else {
        /* ---- Père : nombres impairs 1, 3, 5, ..., 19 ---- */
        for (int i = 1; i <= 19; i += 2) {
            /* Attendre que le fils ait affiché son nombre */
            while (ATTENTE_PERE) ; /* attente active */
            /* Réarmer avant de signaler le fils */
            ATTENTE_PERE = 1;
        }
    }
}
```

```

        printf("%d ", i);
        fflush(stdout);
        /* Signaler au fils qu'il peut continuer */
        kill(pidFork, SIGUSR2);
    }
    /* Attendre le dernier SIGUSR1 du fils (confirmation affichage de 20) */
    while (ATTENTE_PERE) ;
    printf("\n");
    wait(NULL);
}

return 0;
}

```

B.4 - Version 4 : attente passive avec sigsuspend (exercice 5.4)

Remplacement des boucles while() par sigsuspend(). Construction des masques de signaux et blocage préventif de SIGUSR2 dans le fils pour éliminer la race condition.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>

/* Handlers vides : leur seul rôle est d'interrompre sigsuspend().
 * sigsuspend() se réveille à la réception de n'importe quel signal
 * non bloqué et dont le handler ne termine pas le processus. */
void handler_sigusr1(int sig) { }
void handler_sigusr2(int sig) { }

int main() {
    pid_t pidFork;
    sigset_t mask_usr1; /* masque pour le père : tout bloqué sauf SIGUSR1 */
    sigset_t mask_usr2; /* masque pour le fils : tout bloqué sauf SIGUSR2 */
    sigset_t block_usr2; /* ensemble utilisé pour bloquer SIGUSR2 dans le fils */

    signal(SIGUSR1, handler_sigusr1);
    signal(SIGUSR2, handler_sigusr2);

    /* Masque père : tous signaux bloqués SAUF SIGUSR1 */
    sigfillset(&mask_usr1);
    sigdelset(&mask_usr1, SIGUSR1);

    /* Masque fils : tous signaux bloqués SAUF SIGUSR2 */
    sigfillset(&mask_usr2);
    sigdelset(&mask_usr2, SIGUSR2);

    /* Préparer le masque de blocage de SIGUSR2 dans le fils */
    sigemptyset(&block_usr2);
    sigaddset(&block_usr2, SIGUSR2);

    pidFork = fork();

    if (pidFork == 0) {
        /* Bloquer SIGUSR2 dans le fils dès le début pour éviter qu'un signal
         * arrive entre kill(getppid(), SIGUSR1) et sigsuspend(&mask_usr2).
         * Sans ce blocage, le signal pourrait être perdu (race condition). */
        sigprocmask(SIG_BLOCK, &block_usr2, NULL);

        /* ---- Fils : nombres pairs 0, 2, 4, ..., 20 ---- */
        for (int i = 0; i <= 20; i += 2) {
            printf("%d ", i);
            sleep(1);
            fflush(stdout);
            /* Débloquer le père */
            kill(getppid(), SIGUSR1);
            /* Attente passive : endort le processus jusqu'à réception de SIGUSR2 */
            if (i < 20)
                sigsuspend(&mask_usr2);
        }
    } else {
        /* ---- Père : nombres impairs 1, 3, 5, ..., 19 ---- */
        for (int i = 1; i <= 19; i += 2) {
            /* Attente passive jusqu'à SIGUSR1 du fils */
            sigsuspend(&mask_usr1);
            printf("%d ", i);
            sleep(1);
            fflush(stdout);
            /* Débloquer le fils */
            kill(pidFork, SIGUSR2);
        }
    }
}
```

```
    }  
    /* Attendre le dernier SIGUSR1 (fils a affiché 20) */  
    sigsuspend(&mask_usr1);  
    printf("\n");  
    wait(NULL);  
}  
  
return 0;  
}
```


Annexe C - TP6 : Threads et l'orchestre

C.1 - Version 1 : programme initial sans synchronisation (exercice 6.2)

Création des N+1 threads avec pthread. Aucune règle de synchronisation : l'orchestre et les spectateurs s'exécutent librement. Sert de base pour les versions suivantes.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define N 6 /* nombre de spectateurs */
#define ORCHESTRA_DURATION 10 /* durée du concert en secondes */
#define SPECTATOR_DURATION 5 /* temps passé par chaque spectateur */

/* Fonction exécutée par le thread orchestre */
void *orchestra(void *arg)
{
    printf("Orchestre commence a jouer\n");
    sleep(ORCHESTRA_DURATION);
    printf("Orchestre a fini de jouer\n");
    return NULL;
}

/* Fonction exécutée par chaque thread spectateur
 * arg : pointeur vers l'identifiant du spectateur (int) */
void *spectateur(void *arg)
{
    int id = *(int *)arg;

    printf("Spectateur %d entre dans la salle de concert\n", id);
    sleep(SPECTATOR_DURATION); /* simule le temps passé dans la salle */
    printf("Spectateur %d sort de la salle de concert\n", id);

    return NULL;
}

int main(void)
{
    pthread_t orchestra_thread;
    pthread_t spectateur_threads[N];
    int ids[N];

    /* Créer le thread orchestre */
    pthread_create(&orchestra_thread, NULL, orchestra, NULL);

    /* Créer les N threads spectateurs avec 1 seconde de décalage
     * pour distinguer les entrées dans la sortie */
    for (int i = 0; i < N; i++) {
        ids[i] = i + 1;
        pthread_create(&spectateur_threads[i], NULL, spectateur, &ids[i]);
        sleep(1);
    }

    /* Attendre la fin du thread orchestre */
    pthread_join(orchestra_thread, NULL);
    /* Attendre la fin de chaque thread spectateur */
    for (int i = 0; i < N; i++)
        pthread_join(spectateur_threads[i], NULL);

    return 0;
}
```

C.2 - Version 2 : comptage avec sémaphore mutex VERROU (exercice 6.3)

Ajout de la variable partagée CPT et du sémaphore mutex VERROU pour afficher le nombre de spectateurs dans la salle de façon cohérente. La section critique est minimisée.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 6
#define ORCHESTRA_DURATION 10
#define SPECTATOR_DURATION 5

/* Variable partagée : nombre de spectateurs actuellement dans la salle */
int CPT = 0;
/* Sémaphore mutex : protège l'accès concurrent à CPT (initialisé à 1) */
sem_t VERROU;

void *orchestra(void *arg)
{
    printf("Orchestre commence a jouer\n");
    sleep(ORCHESTRA_DURATION);
    printf("Orchestre a fini de jouer\n");
    return NULL;
}

void *spectateur(void *arg)
{
    int id = *(int *)arg;
    int local; /* copie locale pour éviter de garder le verrou pendant printf */

    /* Section critique : incrémenter CPT de façon atomique */
    sem_wait(&VERROU);
    local = ++CPT;
    sem_post(&VERROU);
    /* Afficher APRES avoir relâché le verrou pour minimiser la section critique */
    printf("Spectateur %d entre dans la salle de concert (%d spectateur(s) dans la
salle)\n", id, local);

    sleep(SPECTATOR_DURATION);

    /* Section critique : décrémenter CPT de façon atomique */
    sem_wait(&VERROU);
    local = --CPT;
    sem_post(&VERROU);
    printf("Spectateur %d sort de la salle de concert (%d spectateur(s) dans la
salle)\n", id, local);

    return NULL;
}

int main(void)
{
    pthread_t orchestra_thread;
    pthread_t spectateur_threads[N];
    int ids[N];

    /* Initialiser VERROU en tant que mutex local (0) avec valeur initiale 1 */
    sem_init(&VERROU, 0, 1);

    pthread_create(&orchestra_thread, NULL, orchestra, NULL);
    for (int i = 0; i < N; i++) {
        ids[i] = i + 1;
        pthread_create(&spectateur_threads[i], NULL, spectateur, &ids[i]);
    }
}
```

```
        sleep(1);
    }
    pthread_join(orchestra_thread, NULL);
    for (int i = 0; i < N; i++)
        pthread_join(spectateur_threads[i], NULL);

    sem_destroy(&VERROU);
    return 0;
}
```

C.3 - Version 3 : ajout de DEB_CONCERT pour le démarrage conditionnel (exercice 6.4)

Ajout du sémaphore DEB_CONCERT initialisé à 0. L'orchestre est bloqué tant que NMIN spectateurs ne sont pas présents. Le Nième spectateur débloquent l'orchestre avec sem_post().

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 6
#define ORCHESTRA_DURATION 10
#define SPECTATOR_DURATION 5
#define NMIN 3 /* nombre minimum de spectateurs pour démarrer */

int CPT = 0;
sem_t VERROU;
/* Sémaphore de démarrage : l'orchestre attend que NMIN spectateurs soient là.
 * Initialisé à 0 : l'orchestre bloque sur sem_wait() jusqu'au premier sem_post(). */
sem_t DEB_CONCERT;

void *orchestra(void *arg)
{
    /* Bloquer jusqu'à ce que NMIN spectateurs soient présents */
    sem_wait(&DEB_CONCERT);
    printf("Orchestre commence à jouer\n");
    sleep(ORCHESTRA_DURATION);
    printf("Orchestre a fini de jouer\n");
    return NULL;
}

void *spectateur(void *arg)
{
    int id = *(int *)arg;
    int local;

    sem_wait(&VERROU);
    local = ++CPT;
    sem_post(&VERROU);
    printf("Spectateur %d entre dans la salle de concert (%d spectateur(s) dans la
salle)\n", id, local);

    /* Si ce spectateur est le Nième requis, débloquent l'orchestre.
     * On teste 'local' (copie hors section critique) pour ne pas
     * re-acquérir VERROU inutilement. */
    if (local == NMIN)
        sem_post(&DEB_CONCERT);

    sleep(SPECTATOR_DURATION);

    sem_wait(&VERROU);
    local = --CPT;
    sem_post(&VERROU);
    printf("Spectateur %d sort de la salle de concert (%d spectateur(s) dans la
salle)\n", id, local);

    return NULL;
}

int main(void)
{
    pthread_t orchestra_thread;
    pthread_t spectateur_threads[N];
    int ids[N];
```

```

sem_init(&VERROU, 0, 1); /* mutex */
sem_init(&DEB_CONCERT, 0, 0); /* bloquant au départ */

pthread_create(&orchestra_thread, NULL, orchestra, NULL);
for (int i = 0; i < N; i++) {
    ids[i] = i + 1;
    pthread_create(&spectateur_threads[i], NULL, spectateur, &ids[i]);
    sleep(1);
}
pthread_join(orchestra_thread, NULL);
for (int i = 0; i < N; i++)
    pthread_join(spectateur_threads[i], NULL);

sem_destroy(&VERROU);
sem_destroy(&DEB_CONCERT);
return 0;
}

```

C.4 - Version 4 : ajout de FIN_CONCERT, version threads finale (exercice 6.5)

Ajout du sémaphore FIN_CONCERT. L'orchestre libère un jeton en fin de concert, les spectateurs se le passent en cascade (sem_wait puis sem_post). La temporisation de 5 secondes des spectateurs est supprimée.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 6
#define ORCHESTRA_DURATION 10
#define SPECTATOR_DURATION 5 /* plus utilisé dans cette version */
#define NMIN 3

int CPT = 0;
sem_t VERROU;
sem_t DEB_CONCERT;
/* Sémaphore de fin : l'orchestre poste une fois, les spectateurs
 * se passent le "jeton" en cascade (pattern broadcast par relais). */
sem_t FIN_CONCERT;

void *orchestra(void *arg)
{
    sem_wait(&DEB_CONCERT);
    printf("Orchestre commence a jouer\n");
    sleep(ORCHESTRA_DURATION);
    printf("Orchestre a fini de jouer\n");
    /* Libérer le premier spectateur ; celui-ci repostera pour le suivant */
    sem_post(&FIN_CONCERT);
    return NULL;
}

void *spectateur(void *arg)
{
    int id = *(int *)arg;
    int local;

    sem_wait(&VERROU);
    local = ++CPT;
    sem_post(&VERROU);
    printf("Spectateur %d entre dans la salle de concert (%d spectateur(s) dans la
salle)\n", id, local);

    if (local == NMIN) {
        printf("Le concert peut commencer\n");
        sem_post(&DEB_CONCERT);
    }

    /* Les spectateurs n'ont plus de temporisation propre :
     * ils attendent simplement la fin du concert. */
    /* Attendre la fin du concert (sem_wait bloquant) */
    sem_wait(&FIN_CONCERT);
    /* Propager le jeton au spectateur suivant (relais cascade) */
    sem_post(&FIN_CONCERT);

    sem_wait(&VERROU);
    local = --CPT;
    sem_post(&VERROU);
    printf("Spectateur %d sort de la salle de concert (%d spectateur(s) dans la
salle)\n", id, local);

    return NULL;
}
```

```

int main(void)
{
    pthread_t orchestra_thread;
    pthread_t spectateur_threads[N];
    int ids[N];

    sem_init(&VERROU, 0, 1); /* mutex */
    sem_init(&DEB_CONCERT, 0, 0); /* bloquant au départ */
    sem_init(&FIN_CONCERT, 0, 0); /* bloquant au départ */

    pthread_create(&orchestra_thread, NULL, orchestra, NULL);
    for (int i = 0; i < N; i++) {
        ids[i] = i + 1;
        pthread_create(&spectateur_threads[i], NULL, spectateur, &ids[i]);
        sleep(1);
    }

    pthread_join(orchestra_thread, NULL);
    for (int i = 0; i < N; i++)
        pthread_join(spectateur_threads[i], NULL);

    sem_destroy(&VERROU);
    sem_destroy(&DEB_CONCERT);
    sem_destroy(&FIN_CONCERT);
    return 0;
}

```

C.5 - Version 5 : adaptation avec processus et signaux (exercice 6.6)

Remplacement des threads par deux processus créés avec fork(). Père = orchestre, fils = spectateurs. Synchronisation par SIGUSR1/SIGUSR2 avec sigsuspend(). Les signaux sont bloqués avant le fork() pour éviter toute perte de signal.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

#define N 6
#define ORCHESTRA_DURATION 10
#define NMIN 3

/* Flag volatile sig_atomic_t : type garanti atomique pour les handlers de signaux */
static volatile sig_atomic_t got_signal = 0;

/* Handler générique : on note juste qu'un signal est arrivé */
static void handler(int sig) { (void)sig; got_signal = 1; }

int main(void)
{
    struct sigaction sa;
    sigset_t block_mask; /* signaux bloqués pendant l'initialisation */
    sigset_t wait_mask; /* masque vide : sigsuspend débloquent sur tout signal */

    sa.sa_handler = handler;
    sa.sa_flags = 0;
    sigemptyset(&sa.sa_mask);

    /* Bloquer SIGUSR1 et SIGUSR2 AVANT le fork.
     * Objectif : éviter qu'un signal soit délivré entre le fork() et le
     * premier sigsuspend() dans l'un des processus (race condition). */
    sigemptyset(&block_mask);
    sigaddset(&block_mask, SIGUSR1);
    sigaddset(&block_mask, SIGUSR2);
    sigprocmask(SIG_BLOCK, &block_mask, NULL);

    /* Masque vide pour sigsuspend : les deux signaux seront débloqués
     * pendant l'attente et pourront interrompre le sommeil. */
    sigemptyset(&wait_mask);

    pid_t child_pid = fork();
    if (child_pid < 0) { perror("fork"); exit(1); }

    if (child_pid == 0) {
        /* ---- Fils : rôle spectateurs ---- */
        /* Le fils écoute SIGUSR2 (signal "fin de concert" envoyé par le père) */
        sigaction(SIGUSR2, &sa, NULL);

        int cpt = 0;
        for (int i = 1; i <= N; i++) {
            cpt++;
            printf("Spectateur %d entre dans la salle de concert"
                   " (%d spectateur(s) dans la salle)\n", i, cpt);
            /* Quand le NMIn-ième spectateur entre, prévenir le père */
            if (cpt == NMIn)
                kill(getppid(), SIGUSR1);
            if (i < N)
                sleep(1); /* décalage pour simuler les arrivées progressives */
        }

        /* Attendre le signal de fin de concert du père.

```



```

    * got_signal passe à 1 dans le handler quand SIGUSR2 est reçu. */
    while (!got_signal)
        sigsuspend(&wait_mask);

    /* Sortie de tous les spectateurs après le concert */
    for (int i = 1; i <= N; i++) {
        cpt--;
        printf("Spectateur %d sort de la salle de concert"
               " (%d spectateur(s) dans la salle)\n", i, cpt);
    }
    exit(0);

} else {
    /* ---- Père : rôle orchestre ---- */
    /* Le père écoute SIGUSR1 (signal "NMIN spectateurs présents" envoyé par le
fils) */
    sigaction(SIGUSR1, &sa, NULL);

    /* Attendre que le fils signale NMIN spectateurs dans la salle */
    while (!got_signal)
        sigsuspend(&wait_mask);

    printf("Orchestre commence a jouer\n");
    sleep(ORCHESTRA_DURATION);
    printf("Orchestre a fini de jouer\n");

    /* Signaler au fils que le concert est terminé */
    kill(child_pid, SIGUSR2);
    wait(NULL);
}

return 0;
}

```